LAPORAN PRAKTIKUM

STRUKTUR DATA

STACK

PEKAN 3

Disusun Oleh :

KINAYA NOVRYA MANDA

(2511531016)

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

MUHAMMAD GALID AVERO

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis sampaikan kehadirat Allah SWT. Salawat dan salam disampaikan kepada Nabi Muhammad SAW. Karena thaufik dan hidayah-Nya, laporan praktikum ini dapat diselesaikan dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban dari kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai sarana untuk memperdalam pemahaman mengenai konsep dasar serta penerapan bahasa pemrograman Java.

Melalui praktikum ini, penulis memperoleh pengalaman langsung dalam memahami Implementasi Stack menggunakan Java. Diharapkan laporan ini dapat memberikan gambaran yang jelas mengenai materi yang telah dipelajari serta hasil dari percobaan yang dilakukan selama praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi isi maupun penyajiannya. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang.

Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu, asisten laboratorium, serta semua pihak yang telah memberikan bimbingan, arahan, dan dukungan sehingga laporan praktikum ini dapat tersusun. Semoga laporan ini dapat bermanfaat bagi pembaca, khususnya dalam memahami dasar-dasar pemrograman Java.

Padang, April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....i

DAFTAR ISI.....ii

DAFTAR LAMPIRAN.....iii

BAB 1 PENDAHULUAN

1.1 Latar Belakang..... 1

1.2 Tujuan Praktikum.....2

1.3 Manfaat Praktikum.....2

BAB 2 PEMBAHASAN

2.1 Dasar Teori.....3

2.2 Kode Program..... 5

2.3 Langkah Kerja.....9

2.4 Analisis Hasil..... 20

BAB 3 KESIMPULAN

3.1 Hasil Praktikum.....21

3.2 Saran Pengembangan..... 24

DAFTAR KEPUSTAKAAN.....25

TUGAS PRAKTIKUM PEKAN 3.....26

DAFTAR LAMPIRAN

Kode Program 2.1 (Kode Program StackArray).....5

Kode Program 2.2 (Kode Program StackArrayDriver)..... 6

Kode Program 2.3 (Kode Program StackPostfix).....6

Kode Program 2.4 (Kode Program NilaiMaksimum).....7

Kode Program 2.5 (Kode Program SiswaStack).....8

Gambar 2.1 (Langkah 1 StackArray).....9

Gambar 2.2 (Langkah 2 StackArray).....9

Gambar 2.3 (Langkah 3 StackArray).....10

Gambar 2.4 (Langkah 4 StackArray).....10

Gambar 2.5 (Langkah 5 StackArray).....10

Gambar 2.6 (Langkah 6 StackArray)..... 11

Gambar 2.7 (Langkah 7 StackArray)..... 11

Gambar 2.8 (Langkah 1 StackArrayDriver).....11

Gambar 2.9 (Langkah 2 StackArrayDriver)..... 12

Gambar 2.10 (Langkah 3 StackArrayDriver)..... 12

Gambar 2.11 (Langkah 4 StackArrayDriver).....12

Gambar 2.12 (Langkah 1 StackPostfix).....13

Gambar 2.13 (Langkah 2 StackPostfix).....13

Gambar 2.14 (Langkah 3 StackPostfix).....13

Gambar 2.15 (Langkah 4 StackPostfix).....14

Gambar 2.16 (Langkah 5 StackPostfix).....14

Gambar 2.17 (Langkah 6 StackPostfix).....14

Gambar 2.18 (Langkah 1 NilaiMaksimum).....15

Gambar 2.19 (Langkah 2 NilaiMaksimum).....15
Gambar 2.20 (Langkah 3 NilaiMaksimum).....15
Gambar 2.21 (Langkah 4 NilaiMaksimum).....16
Gambar 2.22 (Langkah 5 NilaiMaksimum).....16
Gambar 2.23 (Langkah 6 NilaiMaksimum).....16
Gambar 2.24 (Langkah 7 NilaiMaksimum).....16
Gambar 2.25 (Langkah 1 SiswaStack).....17
Gambar 2.26 (Langkah 2 SiswaStack).....17
Gambar 2.27 (Langkah 3 SiswaStack).....17
Gambar 2.28 (Langkah 4 SiswaStack).....18
Gambar 2.29 (Langkah 5 SiswaStack).....18
Gambar 2.30 (Langkah 6 SiswaStack).....18
Gambar 2.31 (Langkah 7 SiswaStack).....19
Gambar 2.32 (Langkah 8 SiswaStack).....19
Gambar 2.33 (Langkah 9 SiswaStack).....19
Gambar 2.34 (Langkah 10 SiswaStack).....19
Gambar 3.1 (Hasil Praktikum StackArrayDriver)..... 21
Gambar 3.2 (Hasil Praktikum StackPostfix).....21
Gambar 3.3 (Hasil Praktikum NilaiMaksimum).....22
Gambar 3.4 (Hasil Praktikum SiswaStack).....23

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, pemilihan struktur data yang tepat sangat berpengaruh terhadap cara data dikelola dan diproses. Salah satu struktur data dasar yang cukup sering digunakan adalah *stack*. Stack memiliki konsep kerja yang sederhana, yaitu mengikuti prinsip LIFO (*Last In First Out*), di mana data yang terakhir dimasukkan akan menjadi data pertama yang dikeluarkan. Konsep ini sering muncul dalam berbagai kasus, seperti proses undo, pemanggilan fungsi, maupun pengolahan ekspresi.

Melalui materi yang dipelajari, stack tidak hanya dipahami sebagai konsep, tetapi juga sebagai struktur data yang memiliki aturan operasi yang jelas. Operasi seperti push, pop, dan peek menunjukkan bagaimana data ditambahkan, diambil, dan diperiksa dari posisi teratas. Selain itu, kondisi seperti *overflow* dan *underflow* juga menjadi bagian penting yang perlu diperhatikan agar program dapat berjalan dengan benar tanpa kesalahan saat mengelola data.

Dalam praktikum ini, pemahaman mengenai stack diperkuat melalui penjelasan jenis-jenis stack serta cara implementasinya, baik menggunakan array maupun *linked list*. Dengan mempelajari alur kerja dan algoritma dasar dari setiap operasi, praktikum ini membantu memahami bagaimana data dapat dikelola secara terstruktur sesuai dengan prinsip stack. Proses ini juga melatih cara berpikir sistematis dalam menyusun logika program yang berkaitan dengan pengolahan data bertumpuk.

1.2 Tujuan Praktikum

Tujuan dilakukannya praktikum ini adalah sebagai berikut :

1. Memahami konsep dasar struktur data stack beserta prinsip kerjanya (LIFO).
2. Mengetahui dan mengimplementasikan operasi dasar stack seperti push, pop, dan peek dalam pemrograman Java.
3. Mampu menerapkan stack dalam penyelesaian masalah sederhana dalam pemrograman.

1.3 Manfaat Praktikum

Manfaat dilakukannya praktikum ini adalah sebagai berikut :

1. Mahasiswa dapat memahami cara kerja struktur data stack dalam pengelolaan data.
2. Mahasiswa menjadi lebih terampil dalam mengimplementasikan operasi stack dalam pemrograman Java.
3. Mahasiswa terbiasa berpikir logis dan terstruktur dalam menyelesaikan masalah yang berkaitan dengan data bertumpuk dalam program Java..

BAB II
PEMBAHASAN

2.1 Dasar Teori

Stack (tumpukan) merupakan struktur data linier yang memiliki aturan tertentu dalam pengoperasiannya, yaitu mengikuti prinsip LIFO (*Last In First Out*) atau FILO (*First In Last Out*). Artinya, elemen yang terakhir dimasukkan ke dalam stack akan menjadi elemen pertama yang dikeluarkan. Struktur ini banyak digunakan dalam pemrograman karena konsepnya sederhana dan sesuai untuk pengolahan data yang bersifat bertumpuk, seperti pada proses undo, evaluasi ekspresi, dan pemanggilan fungsi.

Berdasarkan jenisnya, stack dibedakan menjadi:

1. Stack ukuran tetap, yaitu stack yang memiliki kapasitas tetap dan tidak dapat bertambah atau berkurang, sehingga dapat mengalami kondisi *overflow* saat penuh dan *underflow* saat kosong,
2. Stack ukuran dinamis, yaitu stack yang ukurannya dapat berubah secara otomatis sesuai kebutuhan, biasanya diimplementasikan menggunakan *linked list*.

Selain itu, terdapat beberapa operasi dasar pada stack, yaitu:

1. push, untuk menambahkan elemen ke dalam stack,
2. pop, untuk menghapus dan mengambil elemen teratas,
3. peek, untuk melihat elemen teratas tanpa menghapusnya,
4. isEmpty, untuk mengecek apakah stack kosong,
5. size, untuk mengetahui jumlah elemen dalam stack.

Proses kerja pada stack juga memiliki langkah-langkah tertentu. Pada operasi push, langkahnya adalah:

1. memeriksa apakah stack penuh,

2. jika penuh maka terjadi *overflow*,
3. jika tidak, nilai top ditambah satu,
4. elemen baru dimasukkan ke posisi teratas.

Sedangkan pada operasi pop, langkahnya adalah:

1. memeriksa apakah stack kosong,
2. jika kosong maka terjadi *underflow*,
3. jika tidak, elemen teratas diambil,
4. nilai top dikurangi satu.

Untuk operasi peek, langkahnya adalah:

1. memeriksa apakah stack kosong,
2. jika kosong maka tidak ada elemen yang dapat ditampilkan,
3. jika tidak, elemen pada posisi top dikembalikan.

Dalam implementasinya, stack dapat dibuat menggunakan array maupun *linked list*. Penggunaan array biasanya digunakan untuk stack ukuran tetap, sedangkan *linked list* lebih cocok untuk stack dinamis. Dengan demikian, struktur data stack sangat berguna dalam pengelolaan data yang membutuhkan mekanisme masuk dan keluar secara berurutan berdasarkan prinsip LIFO.

2.2 Kode Program

2.2.1 StackArray_2511531016

```
package pekan3_2511531016;

public class StackArray_2511531016 {
static final int MAX_1016 = 1000;
int top_1016;
int a_1016[] = new int[MAX_1016];
boolean isEmpty_1016(){
return (top_1016 < 0);
}
StackArray_2511531016(){
top_1016 = -1;
}
boolean push_1016(int x){
if (top_1016 >= (MAX_1016 - 1)) {
System.out.println("Stack Overlow");
return false;
}
else {
a_1016[++top_1016] = x;
System.out.println(x + " dimasukkan dalam stack");
return true;
}
}
int pop_1016(){
if (top_1016 < 0) {
System.out.println("Stack Underflow");
return 0;
}
else {
int x = a_1016[top_1016--];
return x;
}
}
int peek_1016(){
if (top_1016 < 0) {
System.out.println("Stack Underflow");
return 0;
}
else {
int x = a_1016[top_1016];
return x;
}
}
void print () {
for(int i = top_1016;i>-1;i--) {
System.out.print(" "+ a_1016[i]);
}
}
}
```

Kode Program 2.1 (Kode Program StackArray)

2.2.2 StackArrayDriver_2511531016

```
package pekan3_2511531016;

public class StackArrayDriver_2511531016 {
public static void main(String[] args) {
StackArray_2511531016 s = new StackArray_2511531016();
s.push_1016(10);
s.push_1016(20);
s.push_1016(30);
System.out.println(s.pop_1016() + " dikeluarkan dari stack");
System.out.println("Elemen teratas adalah :" + s.peek_1016());
System.out.print("Element pada stack :");
s.print();
}
}
```

Kode Program 2.2 (Kode Program StackArrayDriver)

2.2.3 StackPostfix_2511531016

```
package pekan3_2511531016;
import java.util.Scanner;

public class StackPostfix_2511531016 {
    public static int postfixEvaluate(String expression) {
        Stack<Integer> s_1016 = new Stack<Integer>();
        Scanner input_1016 = new Scanner (expression);
        while (input_1016.hasNext()) {
                if (input_1016.hasNextInt()) {      // an operand (integer)
                        s_1016.push (input_1016.nextInt());
                } else {                            // an operator
                        String operator = input_1016.next();
                        int operand2 = s_1016.pop();
                        int operand1 = s_1016.pop();
                        if (operator.equals("+")) {
                                s_1016.push(operand1 + operand2);
                        } else if (operator.equals("-")) {
                                s_1016.push(operand1 - operand2);
                        } else if (operator.equals("*")) {
                                s_1016.push(operand1 * operand2);
                        } else {
                                s_1016.push(operand1 / operand2);
                        }
                }
        }
        input_1016.close();
        return s_1016.pop();
}
public static void main(String[] args) {
        System.out.println("hasil postfix= " + postfixEvaluate("5 2 4 * + 7 -"));
}
}
```

Kode Program 2.3 (Kode Program StackPostfix)

2.2.4 NilaiMaksimum_2511531016

```
package pekan3_2511531016;
import java.util.Stack;

public class NilaiMaksimum_2511531016 {
public static int max(Stack<Integer> s) {
Stack<Integer> backup = new Stack<Integer>();
int maxValue = s.pop();
backup.push(maxValue);
while (!s.isEmpty()) {
int next = s.pop();
backup.push(next);
maxValue = Math.max(maxValue, next);
}
while (!backup.isEmpty()) {
s.push(backup.pop());
}
return maxValue;
}
public static void main (String[] args) {
Stack<Integer> s = new Stack<Integer>();
s.push(70);
s.push(12);
s.push(20);
System.out.println("isi stack "+s);
System.out.println("Stack Teratas "+s.peek());
System.out.println("Nilai maksimum "+max(s));
}
}
```

Kode Program 2.4 (Kode Program NilaiMaksimum)

2.2.5 SiswaStack_2511531016

```
package pekan3_2511531016;
import java.util.ArrayList;

class Siswa {
String nama;
int nim;
public Siswa(String nama, int nim) {
this.nama = nama;
this.nim = nim;
}
@Override
public String toString() {
return "Nim: " + nim + ", Nama: " + nama;
}
}
public class SiswaStack_2511531016 {
private ArrayList<Siswa> stack;
public SiswaStack_2511531016() {
```

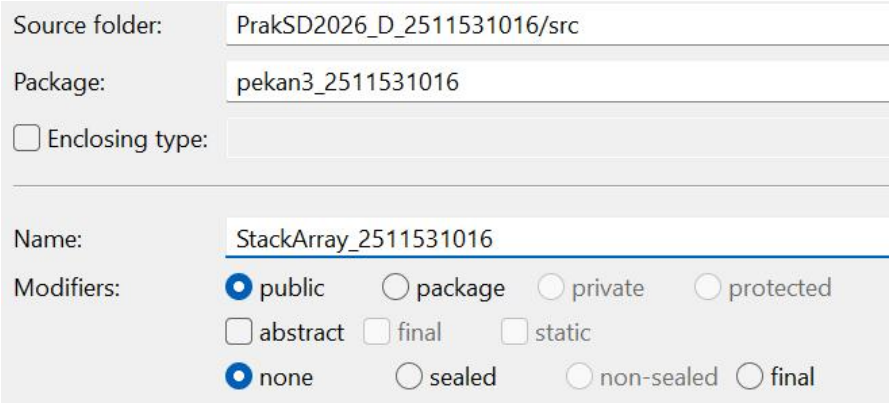
```
stack = new ArrayList<>();
}
public void push(Siswa mhs) {
stack.add(mhs);
}
public Siswa pop() {
if (!isEmpty()) {
return stack.get(stack.size() - 1);
}
return null;
}
public Siswa peek() {
if (!isEmpty()) {
return stack.get(stack.size() - 1);
}
return null;
}
public boolean isEmpty() {
return stack.isEmpty();
}
public void tampilkanSiswa() {
for (int i = stack.size() - 1; i>= 0; i--) {
System.out.println(stack.get(i));
}
}
public static void main(String[] args) {
SiswaStack_2511531016 studentStack = new SiswaStack_2511531016();
Siswa mhs1 = new Siswa("Ali", 1);
Siswa mhs2 = new Siswa("Bobby", 2);
Siswa mhs3 = new Siswa("Charles", 3);
studentStack.push(mhs1);
studentStack.push(mhs2);
studentStack.push(mhs3);
System.out.println("Siswa di dalam stack");
studentStack.tampilkanSiswa();
System.out.println("Siswa teratas " + studentStack.peek());
System.out.println("mengeluarkan siswa teratas dari stack: " +
studentStack.pop());
System.out.println("daftar siswa setelah di pop :");
studentStack.tampilkanSiswa();
}
}
```

Kode Program 2.5 (Kode Program SiswaStack)

2.3 Langkah Kerja

2.3.1 StackArray_2511531016

1. Membuat kelas baru bernama *StackArray_2511531016* dalam *package pekan3_2511531016.*



Gambar 2.1 (Langkah 1 StackArray)

2. Program diawali dengan deklarasi package dan pembuatan class yang mana di dalamnya terdapat variabel top_1016 sebagai penunjuk posisi teratas stack dan array a_1016 sebagai tempat penyimpanan data dengan kapasitas maksimum.

```
package pekan3_2511531016;

public class StackArray_2511531016 {
static final int MAX_1016 = 1000;
int top_1016;
int a_1016[] = new int[MAX_1016];
```

Gambar 2.2 (Langkah 2 StackArray)

3. Selanjutnya, Konstruktor dan pengecekan stack kosong. Konstruktor digunakan untuk menginisialisasi nilai top_1016 menjadi -1 yang menandakan stack masih kosong. Method isEmpty_1016() digunakan untuk mengecek kondisi tersebut.

```
boolean isEmpty_1016()
{
return (top_1016 < 0);
}
StackArray_2511531016()
{
top_1016 = -1;
}
```

Gambar 2.3 (Langkah 3 StackArray)

4. Tambahkan elemen menggunakan method push_1016() ke dalam stack. Sebelum menambah, program mengecek apakah stack penuh(overlow). Jika tidak, elemen ditambahkan ke posisi teratas.

```
boolean push_1016(int x)
{
if (top_1016 >= (MAX_1016 - 1)) {
System.out.println("Stack Overlow");
return false;
}
else {
a_1016[++top_1016] = x;
System.out.println(x + " dimasukkan dalam stack");
return true;
```

Gambar 2.4 (Langkah 4 StackArray)

5. Untuk menghapus elemen teratas, gunakan method pop_1016(). Program akan mengecek apakah stack kosong(underflow). Jika tidak kosong, elemen teratas diambil dan top dikurangi.

```
int pop_1016()
{
if (top_1016 < 0) {
System.out.println("Stack Underflow");
return 0;
}
else {
int x = a_1016[top_1016--];
return x;
```

Gambar 2.5 (Langkah 5 StackArray)

6. Untuk melihat elemen teratas tanpa menghapusnya gunakan method peek_1016(). Jika stack kosong, akan muncul pesan error.

```
int peek_1016()
{
if (top_1016 < 0) {
System.out.println("Stack Underflow");
return 0;
}
else {
int x = a_1016[top_1016];
return x;
```

Gambar 2.6 (Langkah 6 StackArray)

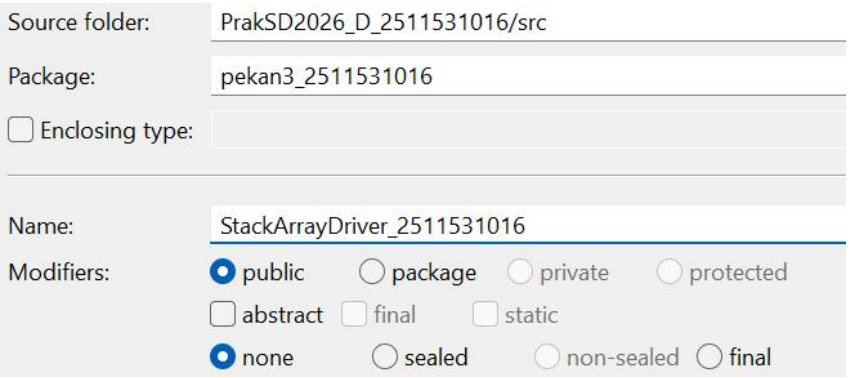
7. Tampilkan seluruh isi stack dari atas ke bawah menggunakan perulangan dan method print().

```
void print () {
for(int i = top_1016;i>-1;i--) {
System.out.print(" "+ a_1016[i]);
}
```

Gambar 2.7 (Langkah 7 StackArray)

2.3.2 StackArrayDriver_2511531016

1. Membuat kelas baru bernama *StackArrayDriver_2511531016* dalam *package pekan3_2511531016.*



Gambar 2.8 (Langkah 1 StackArrayDriver)

2. Deklarasikan package dan class. Di dalam method main, buat objek StackArray_2511531016 sebagai struktur stack. Kemudian tambahkan beberapa elemen ke dalam stack menggunakan method push_1016().

```
package pekan3_2511531016;

public class StackArrayDriver_2511531016 {
public static void main(String[] args) {
StackArray_2511531016 s = new StackArray_2511531016();
s.push_1016(10);
s.push_1016(20);
s.push_1016(30);
```

Gambar 2.9 (Langkah 2 StackArrayDriver)

3. Selanjutnya, untuk menghapus elemen teratas gunakan method pop_1016() dan tampilkan hasilnya. Lalu, tampilkan elemen teratas yang tersisa menggunakan method peek_1016().

```
System.out.println(s.pop_1016() + " dikeluarkan dari stack");
System.out.println("Elemen teratas adalah :" + s.peek_1016());
```

Gambar 2.10 (Langkah 3 StackArrayDriver)

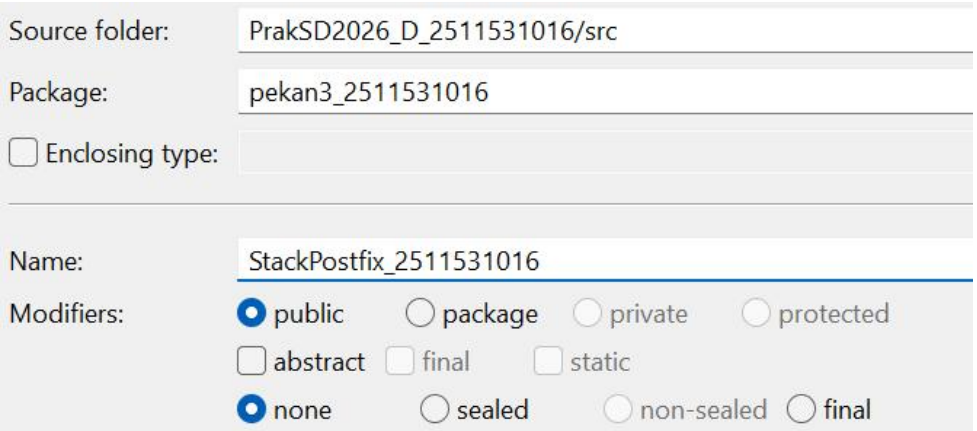
4. Kemudian, tampilkan seluruh isi stack menggunakan method print(), lalu tutup blok program.

```
  System.out.print("Element pada stack :");
  s.print();
  }
  }
```

Gambar 2.11 (Langkah 4 StackArrayDriver)

2.3.3 StackPostfix_2511531016

1. Membuat kelas baru bernama *StackPostfix_2511531016* dalam *package pekan3_2511531016.*



Gambar 2.12 (Langkah 1 StackPostfix)

2. Program dimulai dengan deklarasi package, import Scanner dan Stack, kemudian buat class sebagai pengelola data.

```
package pekan3_2511531016;
import java.util.Scanner;
import java.util.Stack;

public class StackPostfix_2511531016 {
```

Gambar 2.13 (Langkah 2 StackPostfix)

3. Selanjutnya, buat method postfixEvaluate() yang berfungsi untuk menghitung ekspresi postfix menggunakan struktur data stack.

```
public static int postfixEvaluate(String expression) {
Stack<Integer> s_1016 = new Stack<Integer>();
Scanner input_1016 = new Scanner (expression);
```

Gambar 2.14 (Langkah 3 StackPostfix)

4. Program membaca setiap elemen dari ekspresi. Jika berupa angka, maka langsung dimasukkan ke dalam stack menggunakan push().

```
while (input_1016.hasNext()) {
if (input_1016.hasNextInt()) { // an operand (integer)
s_1016.push (input_1016.nextInt());
```

Gambar 2.15 (Langkah 4 StackPostfix)

5. Jika yang dibaca adalah operator, maka dua elemen teratas diambil dari stack, lalu dilakukan operasi sesuai operator, dan hasilnya dimasukkan kembali ke stack.

```
} else { // an operator
String operator = input_1016.next();
int operand2 = s_1016.pop();
int operand1 = s_1016.pop();
if (operator.equals("+")) {
s_1016.push(operand1 + operand2);
} else if (operator.equals("-")) {
s_1016.push(operand1 - operand2);
} else if (operator.equals("*")) {
s_1016.push(operand1 * operand2);
} else {
s_1016.push(operand1 / operand2);
}
}
}
```

Gambar 2.16 (Langkah 5 StackPostfix)

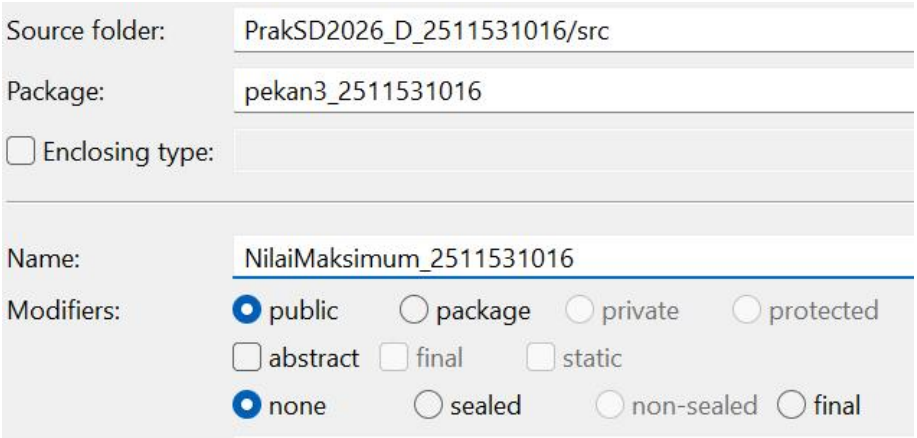
6. Setelah seluruh ekspresi diproses, hasil akhir diambil dari stack dan ditampilkan pada method main.

```
input_1016.close();
return s_1016.pop();
}
public static void main(String[] args) {
System.out.println("hasil postfix= " + postfixEvaluate("5 2 4 * + 7
-"));
}
}
```

Gambar 2.17 (Langkah 6 StackPostfix)

2.3.4 NilaiMaksimum_2511531016

1. Membuat kelas baru bernama *NilaiMaksimum_2511531016* dalam *package pekan3_2511531016.*



Gambar 2.18 (Langkah 1 NilaiMaksimum)

2. Program dimulai dengan deklarasi package, import Stack, kemudian buat class utama.

```
package pekan3_2511531016;
import java.util.Stack;

public class NilaiMaksimum_2511531016 {
```

Gambar 2.19 (Langkah 2 NilaiMaksimum)

3. Selanjutnya, untuk mencari nilai terbesar (nilai maksimum) dalam stack dengan bantuan stack sementara gunakan method max().

```
public static int max(Stack<Integer> s) {
Stack<Integer> backup = new Stack<Integer>();
int maxValue = s.pop();
backup.push(maxValue);
```

Gambar 2.20 (Langkah 3 NilaiMaksimum)

4. Kemudian program membandingkan setiap elemen untuk mendapatkan nilai maksimum.

```
while (!s.isEmpty()) {
int next = s.pop();
backup.push(next);
maxValue = Math.max(maxValue, next);
}
```

Gambar 2.21 (Langkah 4 NilaiMaksimum)

5. Setelah selesai, data dikembalikan ke stack utama.

```
while (!backup.isEmpty()) {
s.push(backup.pop());
}
return maxValue;
}
```

Gambar 2.22 (Langkah 5 NilaiMaksimum)

6. Pada method main, program akan membuat stack dan menambahkan beberapa nilai menggunakan s.push().

```
public static void main (String[] args) {
Stack<Integer> s = new Stack<Integer>();
s.push(70);
s.push(12);
s.push(20);
```

Gambar 2.23 (Langkah 6 NilaiMaksimum)

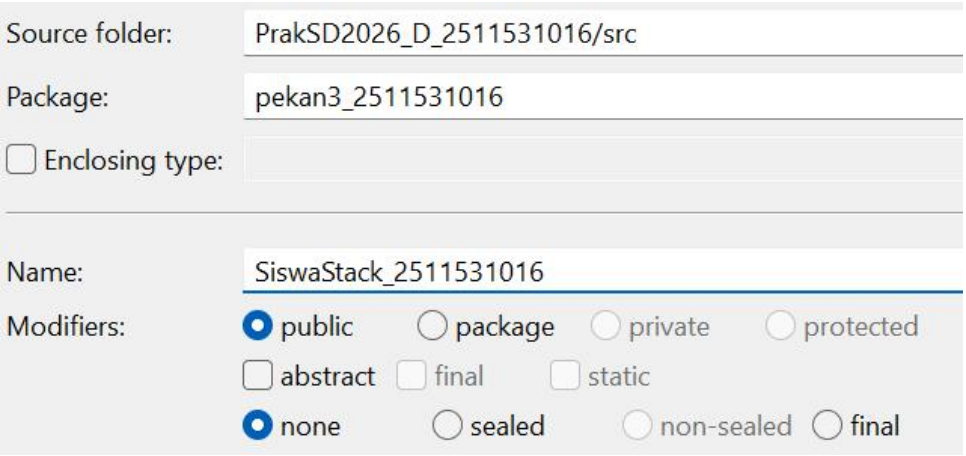
7. Terakhir, tampilkan isi stack, elemen teratas dan nilai maksimum menggunakan System.out.println.

```
System.out.println("isi stack "+s);
System.out.println("Stack Teratas "+s.peek());
System.out.println("Nilai maksimum "+max(s));
}
```

Gambar 2.24 (Langkah 7 NilaiMaksimum)

2.3.5 SiswaStack_2511531016

1. Membuat kelas baru bernama *SiswaStack_2511531016* dalam *package pekan3_2511531016.*



Gambar 2.25 (Langkah 1 SiswaStack)

2. Program dimulai dengan deklarasi package, import ArrayList, serta pembuatan class Siswa yang digunakan untuk merepresentasikan data siswa.

```
package pekan3_2511531016;
import java.util.ArrayList;

class Siswa {
```

Gambar 2.26 (Langkah 2 SiswaStack)

3. Class Siswa memiliki atribut nama dan nim, serta konstruktor untuk menginisialisasi data.

```
String nama;
int nim;
public Siswa(String nama, int nim) {
this.nama = nama;
this.nim = nim;
}
```

Gambar 2.27 (Langkah 3 SiswaStack)

4. Lalu, tampilkan data siswa dalam bentuk string menggunakan method toString().

```
@Override
public String toString() {
return "Nim: " + nim + ", Nama: " + nama;
}
```

Gambar 2.28 (Langkah 4 SiswaStack)

5. Selanjutnya, deklarasikan class SiswaStack_2511531016 dengan atribut ArrayList<Siswa> sebagai penyimpanan data stack.

```
public class SiswaStack_2511531016 {
private ArrayList<Siswa> stack;
public SiswaStack_2511531016() {
stack = new ArrayList<>();
```

Gambar 2.29 (Langkah 5 SiswaStack)

6. Buat method operasi dasar stack, yaitu push() digunakan untuk menambahkan data, pop() untuk mengambil data teratas, peek() untuk melihat data teratas, dan isEmpty() untuk mengecek apakah stack kosong.

```
public void push(Siswa mhs) {
stack.add(mhs);
}
public Siswa pop() {
if (!isEmpty()) {
return stack.get(stack.size() - 1);
}
return null;
}
public Siswa peek() {
if (!isEmpty()) {
return stack.get(stack.size() - 1);
}
return null;
}
public boolean isEmpty() {
return stack.isEmpty();
}
```

Gambar 2.30 (Langkah 6 SiswaStack)

7. Lalu, tampilkan seluruh data siswa dari atas ke bawah menggunakan method tampilkanSiswa().

```
public void tampilkanSiswa() {
for (int i = stack.size() - 1; i>= 0; i--) {
System.out.println(stack.get(i));
```

Gambar 2.31 (Langkah 7 SiswaStack)

8. Selanjutnya, inisialisasi objek dan pada method main, program membuat objek stack serta beberapa objek Siswa.

```
public static void main(String[] args) {
SiswaStack_2511531016 studentStack = new SiswaStack_2511531016();
Siswa mhs1 = new Siswa("Ali", 1);
Siswa mhs2 = new Siswa("Bobby", 2);
Siswa mhs3 = new Siswa("Charles", 3);
```

Gambar 2.32 (Langkah 8 SiswaStack)

9. Data siswa dimasukkan ke dalam stack menggunakan push(), kemudian tampilkan seluruh isi stack.

```
studentStack.push(mhs1);
studentStack.push(mhs2);
studentStack.push(mhs3);
System.out.println("Siswa di dalam stack");
studentStack.tampilkanSiswa();
```

Gambar 2.33 (Langkah 9 SiswaStack)

10. Terakhir, tampilkan elemen teratas menggunakan peek(), hapus elemen teratas dengan pop(), lalu tampilkan kembali isi stack.

```
System.out.println("Siswa teratas " + studentStack.peek());
System.out.println("mengeluarkan siswa teratas dari stack: " +
studentStack.pop());
System.out.println("daftar siswa setelah di pop :");
studentStack.tampilkanSiswa();
```

Gambar 2.34 (Langkah 10 SiswaStack)

2.4 Analisis Hasil

Berdasarkan hasil praktikum yang telah dilakukan, penerapan struktur data *stack* pada beberapa program menunjukkan bahwa konsep LIFO (*Last In First Out*) dapat diimplementasikan dengan baik. Hal ini terlihat pada program seperti StackArray, StackPostfix, hingga SiswaStack, di mana elemen terakhir yang dimasukkan akan menjadi elemen pertama yang diambil. Operasi dasar seperti push, pop, dan peek berjalan sesuai dengan teori, sehingga membantu memahami cara kerja stack tidak hanya secara konsep, tetapi juga melalui implementasi langsung dalam program.

Selain itu, variasi implementasi stack memberikan pemahaman yang lebih luas. Penggunaan array menunjukkan bagaimana pengelolaan data dilakukan secara manual, sedangkan penggunaan Stack bawaan Java dan ArrayList memberikan cara yang lebih praktis dan fleksibel. Pada program evaluasi postfix, terlihat bahwa stack juga dapat digunakan untuk menyelesaikan perhitungan ekspresi secara sistematis. Hal ini menunjukkan bahwa stack tidak hanya berfungsi sebagai penyimpan data, tetapi juga memiliki peran penting dalam penyelesaian masalah dalam pemrograman.

BAB III
KESIMPULAN

3.1 Hasil Praktikum

3.1.1 StackArrayDriver_2511531016

```
Problems  @ Javadoc  Declaration  Console ×
<terminated> StackArrayDriver_2511531016 [Java Application]
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah :20
Element pada stack : 20 10
```

Gambar 3.1 (Hasil Praktikum StackArrayDriver)

Output program tersebut menunjukkan proses kerja stack sesuai prinsip LIFO (*Last In First Out*). Pada awalnya, elemen 10, 20, dan 30 dimasukkan ke dalam stack secara berurutan, sehingga 30 berada di posisi teratas. Ketika dilakukan operasi pop, elemen teratas yaitu 30 dikeluarkan dari stack, yang ditunjukkan oleh output "30 dikeluarkan dari stack". Setelah itu, saat dilakukan peek, elemen teratas yang tersisa adalah 20. Terakhir, saat seluruh isi stack ditampilkan, terlihat urutan "20 10" yang menunjukkan bahwa elemen ditampilkan dari atas ke bawah, sesuai dengan kondisi stack setelah satu elemen dihapus.

3.1.2 StackPostfix_2511531016

```
Problems  @ Javadoc  Declaration  Console ×
<terminated> StackPostfix_2511531016 [Java Application] C:\U
hasil postfix= 6
```

Gambar 3.2 (Hasil Praktikum StackPostfix)

Output “hasil postfix = 6” menunjukkan bahwa program berhasil mengevaluasi ekspresi postfix yang diberikan dengan benar menggunakan struktur data stack. Dalam prosesnya, setiap angka dimasukkan ke dalam stack, kemudian ketika operator ditemukan, dua elemen teratas diambil untuk dilakukan operasi sesuai operator tersebut. Proses ini dilakukan secara berulang hingga seluruh ekspresi selesai diproses, dan hasil akhirnya disimpan kembali di dalam stack. Nilai terakhir yang tersisa di stack merupakan hasil akhir perhitungan, yaitu 6, yang kemudian ditampilkan sebagai output program.

3.1.3 NilaiMaksimum_2511531016

```
Problems  @ Javadoc  Declaration  Console ×
<terminated> NilaiMaksimum_2511531016 [Java Application] C
isi stack [70, 12, 20]
Stack Teratas 20
Nilai maksimum 70
```

Gambar 3.3 (Hasil Praktikum NilaiMaksimum)

Output tersebut menunjukkan proses penggunaan stack untuk mencari nilai maksimum dari beberapa data. Pada awalnya ditampilkan isi stack yaitu [70, 12, 20], yang menandakan elemen dimasukkan secara berurutan dengan 20 sebagai elemen teratas. Selanjutnya, output “Stack Teratas 20” menunjukkan hasil dari operasi peek, yaitu mengambil elemen paling atas tanpa menghapusnya. Terakhir, program menampilkan “Nilai maksimum 70”, yang berarti method max() berhasil menelusuri seluruh elemen dalam stack untuk menemukan nilai terbesar, kemudian mengembalikan stack ke kondisi semula setelah proses pencarian selesai.

3.1.4 SiswaStack_2511531016

```
Problems  @ Javadoc  Declaration  Console ×
<terminated> SiswaStack_2511531016 [Java Application] C:\Users\User\.p2\pool\plug
Siswa di dalam stack
Nim: 3, Nama: Charles
Nim: 2, Nama: Bobby
Nim: 1, Nama: Ali
Siswa teratas Nim: 3, Nama: Charles
mengeluarkan siswa teratas dari stack: Nim: 3, Nama: Charles
daftar siswa setelah di pop :
Nim: 3, Nama: Charles
Nim: 2, Nama: Bobby
Nim: 1, Nama: Ali
```

Gambar 3.4 (Hasil Praktikum SiswaStack)

Output tersebut menunjukkan proses penggunaan stack pada data siswa dengan prinsip LIFO (*Last In First Out*). Pada bagian awal, ditampilkan isi stack dari atas ke bawah yaitu Charles, Bobby, dan Ali, yang menunjukkan bahwa Charles adalah elemen terakhir yang dimasukkan. Selanjutnya, saat dilakukan peek, program menampilkan siswa teratas yaitu Charles tanpa menghapusnya. Kemudian, saat dilakukan pop, elemen teratas yang sama yaitu Charles dikeluarkan dari stack. Namun, pada tampilan akhir terlihat bahwa data Charles masih ada di dalam stack, yang menunjukkan bahwa method pop() pada program hanya mengambil nilai teratas tanpa benar-benar menghapusnya dari struktur data, sehingga isi stack tidak berubah setelah operasi tersebut.

3.2 Saran Pengembangan

Program yang telah dibuat masih dapat dikembangkan agar lebih optimal dan mendekati kebutuhan nyata. Salah satu pengembangan yang dapat dilakukan adalah menambahkan validasi, seperti pengecekan kondisi *overflow* dan *underflow* secara lebih lengkap, serta penanganan input yang tidak sesuai. Dengan adanya validasi ini, program akan menjadi lebih aman dan dapat mengurangi kemungkinan terjadinya error saat dijalankan.

Selain itu, dari sisi fitur dan tampilan, program dapat dibuat lebih interaktif dan informatif. Misalnya dengan menerima input langsung dari pengguna, memperbaiki format output agar lebih rapi, atau menambahkan fitur lain seperti pencarian dan pengelolaan data yang lebih kompleks. Pengembangan juga dapat dilakukan dengan mencoba implementasi stack menggunakan struktur lain seperti *linked list*, sehingga dapat membandingkan kelebihan dan kekurangannya. Dengan demikian, pemahaman terhadap stack akan menjadi lebih mendalam dan aplikatif.

DAFTAR PUSTAKA

[1] G. E. Yuliastuti, Rudiansyah, dkk., *Algoritma dan Struktur Data*. Padang: Get Press Indonesia, 2024.

[2] M. Erkamim, I. Abdurrohim, S. Yuliyanti, dkk., *Buku Ajar Algoritma dan Struktur Data*. Jakarta: PT Sonpedia Publishing Indonesia, 2024.

[3] E. Fernando, A. Faza, A. Sudiarjo, dkk., *Pengantar Struktur Data*. Padang: Get Press Indonesia, 2024.

[4] A. El Rayeb, *Fondasi Pemrograman dan Struktur Data (dengan Java)*. Jakarta: PT Sonpedia Publishing Indonesia, 2025

TUGAS PRAKTIKUM PEKAN 3

Simulasi Browser History

1. Deskripsi Tugas

Pada pekan ini, mahasiswa diminta untuk mengimplementasikan konsep Stack (Tumpukan) yang bekerja dengan prinsip LIFO *(Last In, First Out)*. Tema tugas kali ini adalah Simulasi Browser History. Mahasiswa diwajibkan membuat program yang mensimulasikan perpindahan halaman web. Halaman yang terakhir dibuka akan berada di posisi teratas tumpukan, dan menjadi halaman pertama yang dihapus saat tombol "Back" ditekan.

2. Kode Program

2.1 Website_2511531016

```java
package pekan3_2511531016;

public class Website_2511531016 {
    private String judul_1016;
    private String url_1016;

    // Konstruktor
    public Website_2511531016(String judul, String url) {
        this.judul_1016 = judul;
        this.url_1016 = url;
    }

    // Selektor
    public String getJudul_1016() {
        return judul_1016;
    }
    public String getUrl_1016() {
        return url_1016;
    }

    // Mutator
    public void setJudul_1016(String judul) {
        this.judul_1016 = judul;
    }
    public void setUrl_1016(String url) {
        this.url_1016 = url;
    }

    @Override
    public String toString() {
        return "Judul: " + judul_1016 + ", URL: " + url_1016;
    }
}
```

Penjelasan :

Kelas Website_2511531016 digunakan sebagai representasi data untuk satu halaman web dalam sistem browser history. Kelas ini memiliki dua atribut utama, yaitu judul_1016 dan url_1016, yang masing-masing menyimpan informasi judul halaman dan alamat URL. Struktur ini dibuat sederhana karena memang difokuskan untuk mendukung kebutuhan simulasi stack, yaitu menyimpan riwayat halaman yang dikunjungi. Dengan menggunakan objek sebagai penyimpan data, informasi yang dikelola menjadi lebih terstruktur dibandingkan jika hanya menggunakan tipe data sederhana.

Dalam pengelolaan datanya, kelas ini menyediakan konstruktor yang berfungsi untuk menginisialisasi nilai atribut saat objek dibuat. Selain itu, terdapat method getter seperti getJudul_1016() dan getUrl_1016() yang digunakan untuk mengambil nilai atribut, serta setter yang memungkinkan perubahan data jika diperlukan. Method toString() berperan penting dalam menampilkan objek ke dalam bentuk teks yang mudah dibaca. Dengan adanya method ini, setiap objek dapat langsung dicetak tanpa perlu mengakses atribut satu per satu, sehingga mempermudah proses penampilan data pada program utama.

2.2 Browser_2511531016

```java
package pekan3_2511531016;
import java.util.Scanner;
import java.util.Stack;

public class Browser_2511531016 {
    public static void main(String[] args) {
        Stack<Website_2511531016> history = new Stack<>();
        Scanner sc = new Scanner(System.in);
        int choice;

        do {
            System.out.println("\n=== Browser History NIM: 2511531016 ===");
            System.out.println("1. Kunjungi Website (Push)");
            System.out.println("2. Tombol Back (Pop)");
            System.out.println("3. Lihat Halaman Aktif (Peek)");
            System.out.println("4. Cek Status History");
            System.out.println("5. Keluar");
            System.out.print("Pilihan: ");
            choice = sc.nextInt();
            sc.nextLine();

            switch (choice) {
                case 1:
                    // Push
                    System.out.print("Masukkan Judul: ");
                    String judul = sc.nextLine();
                    System.out.print("Masukkan URL: ");
                    String url = sc.nextLine();

                    history.push(new Website_2511531016(judul, url));
                    System.out.println("Berhasil mengunjungi halaman!");
                    break;


                case 2:
                    // Pop
                    if (!history.isEmpty()) {
                        Website_2511531016 removed = history.pop();
                        System.out.println("Kembali dari: " + removed.getJudul_1016());
                    } else {
                        System.out.println("History kosong.");
                    }
                    break;

                case 3:
                    // Peek
                    if (!history.isEmpty()) {
                        System.out.println("Halaman aktif: " + history.peek());
                    } else {
                        System.out.println("Tidak ada halaman aktif.");
                    }
                    break;

                case 4:
                    // Status
                    System.out.println("Jumlah history: " + history.size());
                    if (history.isEmpty()) {
                        System.out.println("History kosong.");
                    }
                    break;

                case 5:
                    System.out.println("Keluar dari program.");
                    break;

                default:
                    System.out.println("Pilihan tidak valid.");
            }

        } while (choice != 5);

        sc.close();
    }
}
```

Penjelasan :

Kelas Browser_2511531016 berfungsi sebagai pengendali utama jalannya program sekaligus sebagai penghubung antara pengguna dan struktur data yang digunakan. Pada kelas ini digunakan Stack<Website_2511531016> untuk menyimpan riwayat halaman yang dikunjungi. Pemilihan struktur stack didasarkan pada konsep LIFO (Last In First Out), yang sesuai dengan mekanisme browser ketika pengguna menekan tombol kembali (back). Program dijalankan dengan menggunakan perulangan menu sehingga pengguna dapat melakukan berbagai operasi secara berulang hingga memilih untuk keluar.

Setiap menu yang tersedia merepresentasikan operasi dasar pada stack. Menu kunjungi website menggunakan method push() untuk menambahkan data ke dalam stack. Menu tombol back memanfaatkan method pop() untuk menghapus dan mengambil elemen teratas, dengan tambahan pengecekan menggunakan isEmpty() untuk menghindari kesalahan saat stack kosong. Menu lihat halaman aktif menggunakan peek() untuk menampilkan elemen teratas tanpa menghapusnya, sedangkan menu cek status history menggunakan size() untuk mengetahui jumlah data yang tersimpan serta isEmpty() untuk memeriksa kondisi stack. Dengan demikian, semua operasi penting stack sudah dipakai di sini, tapi dikemas dalam bentuk yang lebih realistis seperti penggunaan browser sehari-hari.

3. Output Kode Program

```
Problems  @ Javadoc  Declaration  Console ×
<terminated> Browser_2511531016 [Java Application] C:\Users\User\.p2\pool\plugins\org.eclipse
=== Browser History NIM: 2511531016 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 1
Masukkan Judul: Ilearn UNAND
Masukkan URL: it.ilearn.unand.ac.id
Berhasil mengunjungi halaman!

=== Browser History NIM: 2511531016 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 1
Masukkan Judul: Pengumuman SNBP
Masukkan URL: pengumuman.snbp.snpmb.id
Berhasil mengunjungi halaman!

=== Browser History NIM: 2511531016 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 3
Halaman aktif: Judul: Pengumuman SNBP, URL: pengumuman.snbp.snpmb.id
```

```
=== Browser History NIM: 2511531016 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 4
Jumlah history: 2

=== Browser History NIM: 2511531016 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 2
Kembali dari: Pengumuman SNBP

=== Browser History NIM: 2511531016 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 3
Halaman aktif: Judul: Ilearn UNAND, URL: it.ilearn.unand.ac.id

=== Browser History NIM: 2511531016 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 4
Jumlah history: 1
```

```
=== Browser History NIM: 2511531016 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 5
Keluar dari program.
```

Penjelasan :

Output tersebut memperlihatkan alur penggunaan browser history yang mengikuti konsep stack secara bertahap. Pada awalnya, pengguna menambahkan dua halaman, yaitu "Ilearn UNAND" dan "Pengumuman SNBP". Karena menggunakan prinsip LIFO (Last In First Out), halaman terakhir yang dimasukkan yaitu "Pengumuman SNBP" menjadi halaman aktif. Hal ini terlihat saat menu Peek dipilih, di mana program menampilkan halaman tersebut sebagai halaman teratas. Ketika status history dicek, jumlah data yang tersimpan adalah dua, sesuai dengan jumlah halaman yang telah dikunjungi.

Selanjutnya, ketika pengguna memilih menu Pop, halaman terakhir yaitu "Pengumuman SNBP" dikeluarkan dari stack, yang ditunjukkan oleh pesan "Kembali dari: Pengumuman SNBP". Setelah itu, saat dilakukan pengecekan kembali dengan Peek, halaman aktif berubah menjadi "Ilearn UNAND", karena halaman tersebut kini berada di posisi teratas setelah proses penghapusan. Jumlah history pun berkurang menjadi satu saat dicek kembali. Secara keseluruhan, output ini menunjukkan bahwa mekanisme stack berjalan dengan benar, di mana elemen terakhir yang masuk akan menjadi elemen pertama yang keluar.